Week 5

1. What is the fundamental processing model used by Spark Streaming?

A) Record-at-a-time processing
B) Micro-batch processing
C) Batch-only processing
D) Pipeline processing

Answer: B

Explanation

A. Record-at-a-time processing (Incorrect): Traditional Spark Streaming uses micro-batches rather than processing each record individually.

B. Micro-batch processing (Correct): Spark Streaming processes incoming data streams by dividing them into small batches (micro-batches) and processing each batch using the Spark engine.

C. Batch-only processing (Incorrect): Spark Streaming is designed for stream processing, not just traditional offline batch processing.

D. Pipeline processing (Incorrect): While Spark applications may use processing pipelines, the core Spark Streaming model is based on micro-batches.

2. Which abstraction represents a stream of data in Spark Streaming?

A) DataFrame
B) RDD
C) DStream
D) Dataset

Answer: C

Explanation

A. DataFrame (Incorrect): DataFrames are structured data abstractions used in Spark SQL, not the original streaming abstraction in Spark Streaming.

B. RDD (Incorrect): An RDD represents a single distributed dataset, whereas a DStream is a continuous sequence of RDDs over time.

C. DStream (Correct): A DStream (Discretized Stream) is the basic abstraction in Spark Streaming and represents a continuous stream of data as a sequence of RDDs.

D. Dataset (Incorrect): Datasets provide a typed API for structured data processing but are not the fundamental abstraction of traditional Spark Streaming.

3. Which of the following is NOT a supported Spark Streaming input source?

A) Kafka
B) Flume
C) Twitter
D) HDFS Block Replicator

Answer: D

Explanation

A. Kafka (Incorrect): Apache Kafka is one of the most commonly used input sources for Spark Streaming.

B. Flume (Incorrect): Spark Streaming can ingest streaming data from Apache Flume.

C. Twitter (Incorrect): Spark Streaming provides support for consuming real-time Twitter streams through its streaming APIs.

D. HDFS Block Replicator (Correct): HDFS Block Replicator is an internal HDFS mechanism for maintaining block replicas and is not a Spark Streaming data source.

4. Why is maintaining two separate stacks for batch and streaming considered problematic?

A) Reduces throughput
B) Doubles implementation and operational effort
C) Prevents fault tolerance
D) Eliminates scalability

Answer: B

Explanation

A. Reduces throughput (Incorrect): Having separate stacks does not necessarily reduce throughput; the main issue is increased development and operational overhead.

B. Doubles implementation and operational effort (Correct): Maintaining separate batch and streaming systems requires duplicate code, infrastructure, maintenance, monitoring, and expertise, increasing complexity and cost.

C. Prevents fault tolerance (Incorrect): Both batch and streaming systems can be designed to be fault-tolerant.

D. Eliminates scalability (Incorrect): Separate stacks can still scale; the challenge is managing and maintaining two different processing architectures.

5. What is the main advantage of Spark Streaming's micro-batch architecture?

A) Eliminates RDDs
B) Supports only offline processing
C) Provides fault tolerance and scalability
D) Requires no cluster

Answer: C

Explanation

A. Eliminates RDDs (Incorrect): Spark Streaming is built on RDDs (through DStreams), so it does not eliminate them.

B. Supports only offline processing (Incorrect): Spark Streaming is specifically designed for near real-time stream processing, not just offline batch processing.

C. Provides fault tolerance and scalability (Correct): Spark Streaming processes data in micro-batches using RDDs, enabling fault tolerance through lineage and scalable distributed processing across a cluster.

D. Requires no cluster (Incorrect): Spark Streaming is typically deployed on clusters to achieve scalability and distributed processing.

6. Which Spark Streaming feature helps recover lost data after worker failure?

A) Replicated input data and RDD lineage
B) ZooKeeper election
C) Kafka partitions
D) SQL optimizer

Answer: A

Explanation

A. Replicated input data and RDD lineage (Correct): Spark Streaming recovers lost data using replicated input data sources and RDD lineage information to recompute lost partitions after failures.

B. ZooKeeper election (Incorrect): ZooKeeper handles coordination and leader election in distributed systems but is not Spark Streaming's primary recovery mechanism.

C. Kafka partitions (Incorrect): Kafka partitions help with data distribution and durability, but Spark Streaming recovery mainly relies on lineage and replicated data.

D. SQL optimizer (Incorrect): The SQL optimizer improves query execution plans and has no role in recovering lost streaming data.

7. Which function maintains arbitrary state across batches?

A) updateStateByKey()
B) reduceByKey()
C) union()
D) collect()

Answer: A

Explanation

A. updateStateByKey() (Correct): `updateStateByKey()` maintains and updates state information across multiple batches, enabling stateful stream processing.

B. reduceByKey() (Incorrect): `reduceByKey()` aggregates values within a batch but does not preserve state across multiple batches.

C. union() (Incorrect): `union()` combines two DStreams or RDDs but does not maintain any state information.

D. collect() (Incorrect): `collect()` is an action that returns data to the driver and has no role in maintaining state across batches.

8. In Spark Streaming, every interval results in:

A) A new ZooKeeper node
B) An RDD graph computation
C) A new Kafka broker
D) A new DataNode

Answer: B

Explanation

B. An RDD graph computation (Correct): In Spark Streaming, each batch interval creates a new RDD (or set of RDDs), and Spark executes the corresponding computation graph for that batch.

A. A new ZooKeeper node (Incorrect): ZooKeeper is used for coordination in distributed systems and is not created for each Spark Streaming interval.

C. A new Kafka broker (Incorrect): Kafka brokers are long-running server processes and are not created for every streaming batch.

D. A new DataNode (Incorrect): DataNodes are HDFS storage nodes and are unrelated to Spark Streaming batch intervals.

9. Which statement about Spark Streaming fault tolerance is TRUE?

A) Lost data cannot be recovered.
B) Transformations are exactly-once and fault-tolerant
C) Only input data is fault tolerant.
D) Failures require manual data recreation.

Answer: B

Explanation

A. Lost data cannot be recovered (Incorrect): Spark Streaming can recover lost data using replicated input data and RDD lineage.

B. Transformations are exactly-once and fault-tolerant (Correct): Spark Streaming uses RDD lineage and deterministic transformations to recover lost computations, providing fault-tolerant processing semantics.

C. Only input data is fault tolerant (Incorrect): Both input data and transformation computations are fault tolerant in Spark Streaming.

D. Failures require manual data recreation (Incorrect): Spark automatically recomputes lost partitions and recovers from failures without manual intervention.

10. Which requirement is NOT listed for a stream-processing framework in Spark Streaming?

A) Second-scale latency
B) Efficient fault tolerance
C) Scalability
D) Fixed single-node execution

Answer: D

Explanation

A. Second-scale latency (Incorrect): Low latency (typically on the order of seconds) is a key requirement for stream-processing frameworks.

B. Efficient fault tolerance (Incorrect): Stream-processing systems must recover from failures without losing data, making fault tolerance a fundamental requirement.

C. Scalability (Incorrect): Stream-processing frameworks are expected to scale horizontally to handle increasing data volumes and processing demands.

D. Fixed single-node execution (Correct): Spark Streaming is designed for distributed and scalable cluster-based processing, not fixed execution on a single node.